EXPERIMENT NO. 9 (Group B)

Aim: Data Wrangling Problem Statement: Data Wrangling on Real Estate Market Dataset: "RealEstate_Prices.csv" Description: The dataset contains information about housing prices in a specific real estate market. It includes various attributes such as property characteristics, location, sale prices, and other relevant features. The goal is to perform data wrangling to gain insights into the factors influencing housing prices and prepare the dataset for further analysis or modeling.

Name:Aditya Rade Roll.No:BEAD23167

Outputs:

Step 1 — Import the libraries

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

%matplotlib inline

plt.rcParams["figure.figsize"] = (20, 10)
```

Step 2 — Load and inspect the dataset

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("Real estate.csv")

print(df.head())
print("\nShape:", df.shape)
print("\nColumns:")
print(df.columns)

print("\nMissing values:")
print(df.isnull().sum())
```

```
   No  X1 transaction date  X2 house age  \
0   1             2012.917          32.0
1   2             2012.917          19.5
2   3             2013.583          13.3
3   4             2013.500          13.3
4   5             2012.833           5.0

   X3 distance to the nearest MRT station  X4 number of convenience stores  \
0                                84.87882                               10
1                               306.59470                                9
2                               561.98450                                5
3                               561.98450                                5
4                               390.56840                                5

   X5 latitude  X6 longitude  Y house price of unit area
0     24.98298     121.54024                        37.9
1     24.98034     121.53951                        42.2
2     24.98746     121.54391                        47.3
3     24.98746     121.54391                        54.8
4     24.97937     121.54245                        43.1

Shape: (414, 8)

Columns:
Index(['No', 'X1 transaction date', 'X2 house age',
       'X3 distance to the nearest MRT station',
       'X4 number of convenience stores', 'X5 latitude', 'X6 longitude',
       'Y house price of unit area'],
      dtype='object')

Missing values:
No                                      0
X1 transaction date                     0
X2 house age                            0
X3 distance to the nearest MRT station  0
X4 number of convenience stores         0
X5 latitude                             0
X6 longitude                            0
Y house price of unit area              0
dtype: int64
```

Step 2 — Clean column names + missing values

In [3]:

```python
# Clean column names
df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(" ", "_")
    .str.replace(r"[^a-zA-Z0-9_]", "", regex=True)
)

print("Cleaned columns:")
print(df.columns)

# Remove duplicate rows
```

```
df = df.drop_duplicates()

# Handle missing values
for col in df.columns:
    if df[col].dtype == "object":
        df[col] = df[col].fillna(df[col].mode()[0])
    else:
        df[col] = df[col].fillna(df[col].median())

print("\nMissing values after cleaning:")
print(df.isnull().sum())
```

```
Cleaned columns:
Index(['no', 'x1_transaction_date', 'x2_house_age',
       'x3_distance_to_the_nearest_mrt_station',
       'x4_number_of_convenience_stores', 'x5_latitude', 'x6_longitude',
       'y_house_price_of_unit_area'],
      dtype='object')

Missing values after cleaning:
no                                      0
x1_transaction_date                     0
x2_house_age                            0
x3_distance_to_the_nearest_mrt_station    0
x4_number_of_convenience_stores         0
x5_latitude                             0
x6_longitude                            0
y_house_price_of_unit_area              0
dtype: int64
```

Step 3 — Explore, filter and aggregate

In [4]:
```
print("\nDataset information:")
df.info()

print("\nStatistical summary:")
print(df.describe())

# Display categorical columns
print("\nCategorical columns:")
print(df.select_dtypes(include="object").columns)

# Example filtering
# Change the column name below if your dataset uses a different one
for col in df.select_dtypes(include="object").columns:
    print(f"\nValues in {col}:")
    print(df[col].value_counts().head())

# Numerical correlation
print("\nCorrelation:")
print(df.select_dtypes(include=np.number).corr())
```

```
Dataset information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 414 entries, 0 to 413
Data columns (total 8 columns):
 #   Column                                  Non-Null Count  Dtype
---  ------                                  --------------  -----
 0   no                                      414 non-null    int64
 1   x1_transaction_date                     414 non-null    float64
 2   x2_house_age                            414 non-null    float64
 3   x3_distance_to_the_nearest_mrt_station  414 non-null    float64
 4   x4_number_of_convenience_stores         414 non-null    int64
 5   x5_latitude                             414 non-null    float64
 6   x6_longitude                            414 non-null    float64
 7   y_house_price_of_unit_area              414 non-null    float64
dtypes: float64(6), int64(2)
memory usage: 26.0 KB

Statistical summary:
                no  x1_transaction_date  x2_house_age  \
count  414.000000           414.000000    414.000000
mean   207.500000          2013.148971     17.712560
std    119.655756             0.281967     11.392485
min      1.000000          2012.667000      0.000000
25%    104.250000          2012.917000      9.025000
50%    207.500000          2013.167000     16.100000
75%    310.750000          2013.417000     28.150000
max    414.000000          2013.583000     43.800000

       x3_distance_to_the_nearest_mrt_station  \
count                              414.000000
mean                              1083.885689
std                               1262.109595
min                                 23.382840
25%                                289.324800
50%                                492.231300
75%                               1454.279000
max                               6488.021000

       x4_number_of_convenience_stores  x5_latitude  x6_longitude  \
count                       414.000000   414.000000    414.000000
mean                          4.094203    24.969030    121.533361
std                           2.945562     0.012410      0.015347
min                           0.000000    24.932070    121.473530
25%                           1.000000    24.963000    121.528085
50%                           4.000000    24.971100    121.538630
75%                           6.000000    24.977455    121.543305
max                          10.000000    25.014590    121.566270

       y_house_price_of_unit_area
count                  414.000000
mean                    37.980193
std                     13.606488
min                      7.600000
25%                     27.700000
50%                     38.450000
75%                     46.600000
```

```
max                     117.500000

Categorical columns:
Index([], dtype='object')

Correlation:
                                              no  x1_transaction_date  \
no                                     1.000000            -0.048658
x1_transaction_date                   -0.048658             1.000000
x2_house_age                          -0.032808             0.017549
x3_distance_to_the_nearest_mrt_station -0.013573             0.060880
x4_number_of_convenience_stores       -0.012699             0.009635
x5_latitude                           -0.010110             0.035058
x6_longitude                          -0.011059            -0.041082
y_house_price_of_unit_area            -0.028587             0.087491

                                       x2_house_age  \
no                                        -0.032808
x1_transaction_date                        0.017549
x2_house_age                               1.000000
x3_distance_to_the_nearest_mrt_station     0.025622
x4_number_of_convenience_stores            0.049593
x5_latitude                                0.054420
x6_longitude                              -0.048520
y_house_price_of_unit_area                -0.210567

                                       x3_distance_to_the_nearest_mrt_station  \
no                                                                  -0.013573
x1_transaction_date                                                  0.060880
x2_house_age                                                         0.025622
x3_distance_to_the_nearest_mrt_station                               1.000000
x4_number_of_convenience_stores                                     -0.602519
x5_latitude                                                         -0.591067
x6_longitude                                                        -0.806317
y_house_price_of_unit_area                                          -0.673613

                                       x4_number_of_convenience_stores  \
no                                                           -0.012699
x1_transaction_date                                           0.009635
x2_house_age                                                  0.049593
x3_distance_to_the_nearest_mrt_station                       -0.602519
x4_number_of_convenience_stores                               1.000000
x5_latitude                                                   0.444143
x6_longitude                                                  0.449099
y_house_price_of_unit_area                                    0.571005

                                       x5_latitude  x6_longitude  \
no                                       -0.010110     -0.011059
x1_transaction_date                       0.035058     -0.041082
x2_house_age                              0.054420     -0.048520
x3_distance_to_the_nearest_mrt_station   -0.591067     -0.806317
x4_number_of_convenience_stores           0.444143      0.449099
x5_latitude                               1.000000      0.412924
x6_longitude                              0.412924      1.000000
y_house_price_of_unit_area                0.546307      0.523287
```

```
                                        y_house_price_of_unit_area
no                                                       -0.028587
x1_transaction_date                                       0.087491
x2_house_age                                             -0.210567
x3_distance_to_the_nearest_mrt_station                   -0.673613
x4_number_of_convenience_stores                           0.571005
x5_latitude                                               0.546307
x6_longitude                                              0.523287
y_house_price_of_unit_area                                1.000000
```

Step 4 — Encode categorical variables

In [5]:
```
# One-hot encoding of categorical variables
categorical_cols = df.select_dtypes(include="object").columns

df_encoded = pd.get_dummies(
    df,
    columns=categorical_cols,
    drop_first=True
)

print("Encoded dataset:")
print(df_encoded.head())

print("\nNew shape:", df_encoded.shape)
```

```
Encoded dataset:
   no  x1_transaction_date  x2_house_age  \
0   1             2012.917          32.0
1   2             2012.917          19.5
2   3             2013.583          13.3
3   4             2013.500          13.3
4   5             2012.833           5.0

   x3_distance_to_the_nearest_mrt_station  x4_number_of_convenience_stores  \
0                                84.87882                               10
1                               306.59470                                9
2                               561.98450                                5
3                               561.98450                                5
4                               390.56840                                5

   x5_latitude  x6_longitude  y_house_price_of_unit_area
0     24.98298     121.54024                        37.9
1     24.98034     121.53951                        42.2
2     24.98746     121.54391                        47.3
3     24.98746     121.54391                        54.8
4     24.97937     121.54245                        43.1

New shape: (414, 8)
```

Step 5 — Detect and remove numerical outliers

In [6]:
```
numeric_cols = df.select_dtypes(include=np.number).columns

for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
```

```
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR

    df = df[(df[col] >= lower) & (df[col] <= upper)]

print("Shape after outlier removal:", df.shape)
```

```
Shape after outlier removal: (303, 8)
```

Step 6 — Final cleaned dataset

In [8]:
```
print("Final dataset:")
print(df.head())

print("\nFinal shape:", df.shape)

print("\nFinal missing values:")
print(df.isnull().sum())

# Save cleaned dataset
df.to_csv("Real_estate_cleaned.csv", index=False)

print("\nCleaned dataset saved successfully!")
print("Name: Aditya Rade Roll No: BEAD23167")
```

```
Final dataset:
   no  x1_transaction_date  x2_house_age  \
0   1             2012.917          32.0
1   2             2012.917          19.5
2   3             2013.583          13.3
3   4             2013.500          13.3
4   5             2012.833           5.0

   x3_distance_to_the_nearest_mrt_station  x4_number_of_convenience_stores  \
0                                84.87882                               10
1                               306.59470                                9
2                               561.98450                                5
3                               561.98450                                5
4                               390.56840                                5

   x5_latitude  x6_longitude  y_house_price_of_unit_area
0     24.98298     121.54024                        37.9
1     24.98034     121.53951                        42.2
2     24.98746     121.54391                        47.3
3     24.98746     121.54391                        54.8
4     24.97937     121.54245                        43.1

Final shape: (303, 8)

Final missing values:
no                                      0
x1_transaction_date                     0
x2_house_age                            0
x3_distance_to_the_nearest_mrt_station  0
x4_number_of_convenience_stores         0
x5_latitude                             0
x6_longitude                            0
y_house_price_of_unit_area              0
dtype: int64

Cleaned dataset saved successfully!
Name: Aditya Rade Roll No: BEAD23167
```